

Parameter Estimation

In the previous chapters we have assumed that the parameters of the state space model are known, and only the state needs to be estimated. However, in practical models, the parameters are unknown as well. In this chapter we concentrate on three types of method for parameter estimation: optimization-based methods for computing maximum a posteriori (MAP) or maximum likelihood (ML) estimates, expectation-maximization (EM) algorithms for computing the MAP or ML estimates, and Markov chain Monte Carlo (MCMC) methods for generating Monte Carlo approximations of the posterior distributions. We also show how Kalman filters and RTS smoothers, Gaussian filters and smoothers, and particle filters and smoothers can be used for approximating the marginal likelihoods, parameter posteriors, and other quantities needed by the methods.

16.1 Bayesian Estimation of Parameters in State Space Models

The Bayesian way of treating unknown parameters $\theta \in \mathbb{R}^d$ is to model them as random variables with a certain prior distribution $p(\theta)$. A state space model with unknown parameters can be written in the form

$$\begin{aligned}\theta &\sim p(\theta), \\ \mathbf{x}_0 &\sim p(\mathbf{x}_0 \mid \theta), \\ \mathbf{x}_k &\sim p(\mathbf{x}_k \mid \mathbf{x}_{k-1}, \theta), \\ \mathbf{y}_k &\sim p(\mathbf{y}_k \mid \mathbf{x}_k, \theta).\end{aligned}\tag{16.1}$$

A straightforward way to proceed would now be to form the full posterior distribution via Bayes' rule:

$$p(\mathbf{x}_{0:T}, \theta \mid \mathbf{y}_{1:T}) = \frac{p(\mathbf{y}_{1:T} \mid \mathbf{x}_{0:T}, \theta) p(\mathbf{x}_{0:T} \mid \theta) p(\theta)}{p(\mathbf{y}_{1:T})}, \tag{16.2}$$

where the terms $p(\mathbf{x}_{0:T} \mid \boldsymbol{\theta})$ and $p(\mathbf{y}_{1:T} \mid \mathbf{x}_{0:T}, \boldsymbol{\theta})$ can be computed as

$$p(\mathbf{x}_{0:T} \mid \boldsymbol{\theta}) = p(\mathbf{x}_0 \mid \boldsymbol{\theta}) \prod_{k=1}^T p(\mathbf{x}_k \mid \mathbf{x}_{k-1}, \boldsymbol{\theta}),$$

$$p(\mathbf{y}_{1:T} \mid \mathbf{x}_{0:T}, \boldsymbol{\theta}) = \prod_{k=1}^T p(\mathbf{y}_k \mid \mathbf{x}_k, \boldsymbol{\theta}).$$

If we are only interested in the parameters $\boldsymbol{\theta}$, the proper Bayesian way to proceed is to integrate the states out, which gives the marginal posterior of parameters:

$$p(\boldsymbol{\theta} \mid \mathbf{y}_{1:T}) = \int p(\mathbf{x}_{0:T}, \boldsymbol{\theta} \mid \mathbf{y}_{1:T}) \, d\mathbf{x}_{0:T}. \quad (16.3)$$

Unfortunately, computation of this high-dimensional integral is hard and becomes even harder as we obtain more measurements. In this approach we encounter the same computational problem as was discussed in Sections 1.3 and 6.1, which led us to consider optimal filtering and smoothing instead of the straightforward Bayesian approach. Thus it is again advantageous to look at recursive, filtering, and smoothing kinds of solutions.

In this chapter we present methods for parameter estimation that are based on approximating the marginal posterior distribution

$$p(\boldsymbol{\theta} \mid \mathbf{y}_{1:T}) \propto p(\mathbf{y}_{1:T} \mid \boldsymbol{\theta}) p(\boldsymbol{\theta}), \quad (16.4)$$

without explicitly forming the joint posterior distribution of the states and parameters as in Equation (16.2). Instead, we present recursive algorithms for direct computation of the above distribution. For linear state space models, this can be done exactly, and in non-linear and non-Gaussian models we can use Gaussian filtering or particle filtering-based approximations. Once we know how to evaluate the above distribution, we can estimate the parameters, for example, by finding their maximum a posteriori (MAP) estimate or by sampling from the posterior by Markov chain Monte Carlo (MCMC) methods. If direct evaluation of the distribution is not feasible, we can use the expectation-maximization (EM) algorithm to iteratively find the ML or MAP estimate.

16.1.1 Parameter Posterior and Energy Function

The difficult part in Equation (16.4) is the evaluation of the marginal likelihood $p(\mathbf{y}_{1:T} \mid \boldsymbol{\theta})$. The prior distribution can usually be selected such that

it is easy to evaluate. Although evaluation of the normalization constant for the posterior distribution is a difficult problem, its evaluation is usually avoided by Bayesian computational methods, and thus we do not need to worry about it.

The key to recursive computation of the parameter posterior in state space models is the following factorization (often called prediction error decomposition):

$$p(\mathbf{y}_{1:T} \mid \boldsymbol{\theta}) = \prod_{k=1}^T p(\mathbf{y}_k \mid \mathbf{y}_{1:k-1}, \boldsymbol{\theta}), \quad (16.5)$$

where we have denoted $p(\mathbf{y}_1 \mid \mathbf{y}_{1:0}, \boldsymbol{\theta}) \triangleq p(\mathbf{y}_1 \mid \boldsymbol{\theta})$ for notational convenience. Because each of the terms in the above product can be computed recursively, the whole marginal likelihood can be computed recursively as follows.

Theorem 16.1 (Recursion for marginal likelihood of parameters). *The marginal likelihood of parameters is given by Equation (16.5), where the terms in the product can be computed recursively as*

$$p(\mathbf{y}_k \mid \mathbf{y}_{1:k-1}, \boldsymbol{\theta}) = \int p(\mathbf{y}_k \mid \mathbf{x}_k, \boldsymbol{\theta}) p(\mathbf{x}_k \mid \mathbf{y}_{1:k-1}, \boldsymbol{\theta}) d\mathbf{x}_k, \quad (16.6)$$

where $p(\mathbf{y}_k \mid \mathbf{x}_k, \boldsymbol{\theta})$ is the measurement model, and $p(\mathbf{x}_k \mid \mathbf{y}_{1:k-1}, \boldsymbol{\theta})$ is the predictive distribution of the state, which obeys the recursion

$$\begin{aligned} p(\mathbf{x}_k \mid \mathbf{y}_{1:k-1}, \boldsymbol{\theta}) &= \int p(\mathbf{x}_k \mid \mathbf{x}_{k-1}, \boldsymbol{\theta}) p(\mathbf{x}_{k-1} \mid \mathbf{y}_{1:k-1}, \boldsymbol{\theta}) d\mathbf{x}_{k-1}, \\ p(\mathbf{x}_k \mid \mathbf{y}_{1:k}, \boldsymbol{\theta}) &= \frac{p(\mathbf{y}_k \mid \mathbf{x}_k, \boldsymbol{\theta}) p(\mathbf{x}_k \mid \mathbf{y}_{1:k-1}, \boldsymbol{\theta})}{p(\mathbf{y}_k \mid \mathbf{y}_{1:k-1}, \boldsymbol{\theta})}. \end{aligned} \quad (16.7)$$

Note that the latter equations are just the Bayesian filtering Equations (6.11) and (6.12), where we have explicitly written down the parameter dependence.

Proof Due to the conditional independence of the measurements (Property 6.3), we have

$$\begin{aligned} p(\mathbf{y}_k, \mathbf{x}_k \mid \mathbf{y}_{1:k-1}, \boldsymbol{\theta}) &= p(\mathbf{y}_k \mid \mathbf{x}_k, \mathbf{y}_{1:k-1}, \boldsymbol{\theta}) p(\mathbf{x}_k \mid \mathbf{y}_{1:k-1}, \boldsymbol{\theta}) \\ &= p(\mathbf{y}_k \mid \mathbf{x}_k, \boldsymbol{\theta}) p(\mathbf{x}_k \mid \mathbf{y}_{1:k-1}, \boldsymbol{\theta}). \end{aligned} \quad (16.8)$$

Integrating over $\mathbf{x}_k$ gives Equation (16.6). $\square$

The marginal likelihood obtained via Theorem 16.1 can then be substituted into Equation (16.4) to give the marginal posterior distribution of the parameters. However, instead of working with marginal likelihood or marginal posterior explicitly, in parameter estimation it is often convenient to define the unnormalized negative log-posterior or *energy function* as follows.

Definition 16.2 (Energy function).

$$\varphi_T(\boldsymbol{\theta}) = -\log p(\mathbf{y}_{1:T} \mid \boldsymbol{\theta}) - \log p(\boldsymbol{\theta}). \quad (16.9)$$

Remark 16.3. *The definition of energy function thus implies*

$$p(\boldsymbol{\theta} \mid \mathbf{y}_{1:T}) \propto \exp(-\varphi_T(\boldsymbol{\theta})). \quad (16.10)$$

The energy function can be seen to obey the following simple recursion.

Theorem 16.4 (Recursion for energy function). *The energy function defined in Equation (16.9) can be evaluated recursively as follows.*

- Start from $\varphi_0(\boldsymbol{\theta}) = -\log p(\boldsymbol{\theta})$.
- At each step $k = 1, 2, \dots, T$, compute the following:

$$\varphi_k(\boldsymbol{\theta}) = \varphi_{k-1}(\boldsymbol{\theta}) - \log p(\mathbf{y}_k \mid \mathbf{y}_{1:k-1}, \boldsymbol{\theta}), \quad (16.11)$$

where the terms $p(\mathbf{y}_k \mid \mathbf{y}_{1:k-1}, \boldsymbol{\theta})$ can be computed recursively by Theorem 16.1.

Proof The result follows from substituting Equation (16.5) into the definition of the energy function in Equation (16.9) and identifying the terms corresponding to $\varphi_{k-1}(\boldsymbol{\theta})$. $\square$

16.2 Computational Methods for Parameter Estimation

In this section we briefly go through the underlying ideas in ML- and MAP-based parameter estimation and implementation by direct optimization and by the EM algorithm, as well as the basics of Markov chain Monte Carlo (MCMC) methods. There exist many other parameter estimation methods for state space models and for more general statistical models, but here we concentrate on these, because these approaches are the most widely used (probabilistic methods) in the state space context.

16.2.1 Maximum A Posteriori and Laplace Approximations

The *maximum a posteriori* (MAP) estimate is obtained by determining the location of the maximum of the posterior distribution and using it as the point estimate:

$$\hat{\theta}^{\text{MAP}} = \arg \max_{\theta} [p(\theta \mid \mathbf{y}_{1:T})]. \quad (16.12)$$

The MAP estimate can be equivalently computed as the minimum of the error function defined in Equation (16.9):

$$\hat{\theta}^{\text{MAP}} = \arg \min_{\theta} [\varphi_T(\theta)], \quad (16.13)$$

which is usually numerically more stable and easier to compute. The maximum likelihood (ML) estimate of the parameter is a MAP estimate with a formally uniform prior $p(\theta) \propto 1$.

The minimum of the energy function can be computed by using various gradient-free or gradient-based general optimization algorithms (see, e.g., Luenberger and Ye, 2008). However, to be able to use gradient-based optimization, we will need to evaluate the derivatives of the energy function as well. It is possible to find the derivatives in basically two ways (see, e.g., Cappé et al., 2005).

1. By formally differentiating the energy function recursion equations for a particular method. This results in so-called *sensitivity equations*, which can be implemented as additional recursion equations computed along with the filtering.
2. Using *Fisher's identity*, which expresses the gradient of the energy function as an expectation of the derivative of the complete data log likelihood over the smoothing distribution. The advantage of this approach over direct differentiation is that there is no need for an additional recursion.

The disadvantage of the MAP estimate is that it essentially approximates the posterior distribution with the Dirac delta function

$$p(\theta \mid \mathbf{y}_{1:T}) \simeq \delta(\theta - \hat{\theta}^{\text{MAP}}) \quad (16.14)$$

and thus ignores the spread of the distribution completely.

It is also possible to use a Laplace approximation (Gelman et al., 2013), which uses the second derivative (Hessian) of the energy function to form a Gaussian approximation to the posterior distribution:

$$p(\theta \mid \mathbf{y}_{1:T}) \simeq \mathcal{N}(\theta \mid \hat{\theta}^{\text{MAP}}, [\mathbf{H}(\hat{\theta}^{\text{MAP}})]^{-1}), \quad (16.15)$$

where $\mathbf{H}(\hat{\boldsymbol{\theta}}^{\text{MAP}})$ is the Hessian matrix evaluated at the MAP estimate. However, to implement the Laplace approximation, we need to have a method to compute (or approximate) the second order derivatives of the energy function.

16.2.2 Parameter Inference via Markov Chain Monte Carlo

Markov chain Monte Carlo (MCMC) methods (see, e.g., Liu, 2001; Brooks et al., 2011) are a class of algorithms for drawing random variables from a given distribution by simulating a Markov chain that has the desired distribution as its stationary distribution. The methods are particularly suited for simulating samples from Bayesian posterior distributions $p(\boldsymbol{\theta} \mid \mathbf{y}_{1:T})$, because to implement the methods, we only need to know the unnormalized posterior distribution $\tilde{p}(\boldsymbol{\theta} \mid \mathbf{y}_{1:T}) = p(\mathbf{y}_{1:T} \mid \boldsymbol{\theta}) p(\boldsymbol{\theta})$ or equivalently the energy function in Equation (16.9), and knowledge of the normalization constant of the posterior distribution is not required. The usage of MCMC methods in the state space context has been discussed, for example, by Ninness and Henriksen (2010) and Andrieu et al. (2010).

The *Metropolis–Hastings* (MH) algorithm is the most common type of MCMC method. MH uses a *proposal density* $q(\boldsymbol{\theta}^{(i)} \mid \boldsymbol{\theta}^{(i-1)})$ for suggesting new samples $\boldsymbol{\theta}^{(i)}$ given the previous ones $\boldsymbol{\theta}^{(i-1)}$. The algorithm is the following.

Algorithm 16.5 (Metropolis–Hastings). *The Metropolis–Hastings (MH) algorithm consists of the following steps.*

- Draw the starting point $\boldsymbol{\theta}^{(0)}$ from an arbitrary initial distribution.
- For $i = 1, 2, \dots, N$, do

1. Sample a candidate point $\boldsymbol{\theta}^*$ from the proposal distribution:

$$\boldsymbol{\theta}^* \sim q(\boldsymbol{\theta}^* \mid \boldsymbol{\theta}^{(i-1)}). \quad (16.16)$$

2. Evaluate the acceptance probability

$$\alpha_i = \min \left\{ 1, \exp(\varphi_T(\boldsymbol{\theta}^{(i-1)}) - \varphi_T(\boldsymbol{\theta}^*)) \frac{q(\boldsymbol{\theta}^{(i-1)} \mid \boldsymbol{\theta}^*)}{q(\boldsymbol{\theta}^* \mid \boldsymbol{\theta}^{(i-1)})} \right\}. \quad (16.17)$$

3. Generate a uniform random variable $u \sim \text{U}(0, 1)$ and set

$$\boldsymbol{\theta}^{(i)} = \begin{cases} \boldsymbol{\theta}^*, & \text{if } u \leq \alpha_i, \\ \boldsymbol{\theta}^{(i-1)}, & \text{otherwise.} \end{cases} \quad (16.18)$$

The Metropolis algorithm is a commonly used special case of Metropolis–Hastings, where the proposal distribution is symmetric, $q(\boldsymbol{\theta}^{(i-1)} | \boldsymbol{\theta}^{(i)}) = q(\boldsymbol{\theta}^{(i)} | \boldsymbol{\theta}^{(i-1)})$. In this case the acceptance probability reduces to

$$\alpha_i = \min \left\{ 1, \exp(\varphi_T(\boldsymbol{\theta}^{(i-1)}) - \varphi_T(\boldsymbol{\theta}^*)) \right\}. \quad (16.19)$$

The choice of the proposal distribution is crucial for performance of the Metropolis–Hastings method, and finding a good one is a hard task (see, e.g., Liu, 2001; Brooks et al., 2011). Some choices will lead to Metropolis–Hastings methods where the samples are highly correlated, whereas with some choices the rejection rate becomes too high.

One commonly used choice is to use a Gaussian distribution as the proposal distribution,

$$q(\boldsymbol{\theta}^{(i)} | \boldsymbol{\theta}^{(i-1)}) = N(\boldsymbol{\theta}^{(i)} | \boldsymbol{\theta}^{(i-1)}, \boldsymbol{\Sigma}_{i-1}), \quad (16.20)$$

where $\boldsymbol{\Sigma}_{i-1}$ is some suitable covariance matrix. The resulting algorithm is called the random walk Metropolis algorithm, because the transition distribution above defines a Gaussian random walk in parameter space. With this selection of proposal distribution, the challenge is now to find a suitable covariance matrix for the random walk.

One approach to the problem of selection of the covariance matrix is to use *adaptive Markov chain Monte Carlo (AMCMC)* methods where the covariance of the Gaussian proposal in the Metropolis algorithm is automatically adapted during the MCMC run (see, e.g., Haario et al., 1999, 2001; Andrieu and Thoms, 2008; Vihola, 2012). A typical idea in AMCMC methods is to use the covariance of the previously generated samples as an estimate of the actual covariance of the posterior distribution. Given the covariance, it is possible to compute the covariance of the proposal distribution such that it causes an acceptance rate $\bar{\alpha}_*$ that is optimal in some suitable sense. For the random walk Metropolis algorithm, the optimal acceptance rate in certain ideal conditions is $\bar{\alpha}_* = 0.234$ (Roberts and Rosenthal, 2001).

For example, the *robust adaptive Metropolis (RAM)* algorithm of Vihola (2012) is similar to the adaptive Metropolis (AM) algorithm of Haario et al. (2001) except that the adaptation of the covariance $\boldsymbol{\Sigma}_i$ is done in a slightly different way. The algorithm is the following.

Algorithm 16.6 (RAM algorithm). *The RAM algorithm consists of the following steps.*

1. Draw $\boldsymbol{\theta}^{(0)}$ from an initial distribution $p_0(\boldsymbol{\theta})$, and initialize $\mathbf{S}_0$ to be the lower-triangular Cholesky factor of the initial covariance $\boldsymbol{\Sigma}_0$.
2. Sample a candidate point by $\boldsymbol{\theta}^* = \boldsymbol{\theta}_{i-1} + \mathbf{S}_{i-1} \mathbf{r}_i$, where $\mathbf{r}_i \sim N(\mathbf{0}, \mathbf{I})$.
3. Compute the acceptance probability

$$\alpha_i = \min \{1, \exp(\varphi_T(\boldsymbol{\theta}_{i-1}) - \varphi_T(\boldsymbol{\theta}^*))\}. \quad (16.21)$$

4. Sample a uniform random variable u from the uniform distribution $U(0, 1)$.
5. If $u \leq \alpha_i$, set $\boldsymbol{\theta}^{(i)} = \boldsymbol{\theta}^*$. Otherwise set $\boldsymbol{\theta}^{(i)} = \boldsymbol{\theta}^{(i-1)}$.
6. Compute a lower-triangular matrix $\mathbf{S}_i$ with positive diagonal elements satisfying the equation

$$\mathbf{S}_i \mathbf{S}_i^\top = \mathbf{S}_{i-1} \left(\mathbf{I} + \eta_i (\alpha_i - \bar{\alpha}_*) \frac{\mathbf{r}_i \mathbf{r}_i^\top}{\|\mathbf{r}_i\|^2} \right) \mathbf{S}_{i-1}^\top, \quad (16.22)$$

where $\{\eta_i\}_{i \geq 1} \subset (0, 1]$ is an adaptation step size sequence decaying to zero. Although any such sequence will do, Vihola (2012) suggests $\eta_i = i^{-\gamma}$ with a suitable exponent $\gamma \in (1/2, 1]$.

7. Set $i \leftarrow i + 1$, and go to step 2 until the desired number of samples has been generated.

Instead of the random walk Metropolis algorithm with covariance adaptation it is also possible to use the gradient information in the construction of the proposal distribution. This is the idea used in the Hamiltonian Monte Carlo (HMC) or hybrid Monte Carlo (HMC) method (Duane et al., 1987; Neal, 2011). In HMC, the proposal distribution is constructed by simulating a physical system consisting of particles moving under the influence of a potential (the energy function) and heat bath. The gradient of the energy function enters the equations as the force caused by the potential. The HMC method has been applied in the state space context by Mbalawata et al. (2013).

Another commonly used MCMC method is Gibbs' sampling (see, e.g., Liu, 2001; Brooks et al., 2011), which samples components of the parameters one at a time from their conditional distributions given the other parameters. The advantage of this method is that no rejections are needed: the acceptance probability is identically one. However, in order to implement the method it is necessary to be able to generate samples from the conditional distributions of parameters, which is possible only in a restricted class of models. For various other methods the reader is referred to Brooks et al. (2011).

16.2.3 Expectation Maximization

The expectation-maximization (EM) algorithm is a method to iteratively find an ML estimate of the parameters when direct optimization of the posterior distribution (or equivalently, energy function) is not feasible. The algorithm was originally introduced by Dempster et al. (1977), and applications to state space models have been discussed, for example, in Shumway and Stoffer (1982), Roweis and Ghahramani (2001), and Schön et al. (2011). Gaussian smoothing and sigma point-based approximations in an EM context have also been discussed in Väänänen (2012) and Kokkala et al. (2016). Although the EM algorithm was originally an algorithm for computing ML estimates, it can also be easily modified for computation of MAP estimates, as discussed below.

The EM algorithm is based on the result that even when we cannot evaluate the marginal likelihood as such, we are still often able to compute a lower bound for it as follows. Let $q(\mathbf{x}_{0:T})$ be an arbitrary probability density over the states; then we have

$$\log p(\mathbf{y}_{1:T} \mid \boldsymbol{\theta}) \geq F[q(\mathbf{x}_{0:T}), \boldsymbol{\theta}], \quad (16.23)$$

where the functional F is defined as

$$F[q(\mathbf{x}_{0:T}), \boldsymbol{\theta}] = \int q(\mathbf{x}_{0:T}) \log \frac{p(\mathbf{x}_{0:T}, \mathbf{y}_{1:T} \mid \boldsymbol{\theta})}{q(\mathbf{x}_{0:T})} d\mathbf{x}_{0:T}. \quad (16.24)$$

The key idea behind the EM algorithm is that it is possible to maximize the left-hand side of Equation (16.23) by iteratively maximizing the lower bound $F[q(\mathbf{x}_{0:T}), \boldsymbol{\theta}]$. A simple way to do that is the following iteration (Neal and Hinton, 1999).

Algorithm 16.7 (Abstract EM). *The maximization of the lower bound in Equation (16.24) can be done by coordinate ascent as follows.*

- Start from initial guesses $q^{(0)}, \boldsymbol{\theta}^{(0)}$.
- For $n = 0, 1, 2, \dots$ do the following steps:
 1. E-step: Find $q^{(n+1)} = \arg \max_q F[q, \boldsymbol{\theta}^{(n)}]$.
 2. M-step: Find $\boldsymbol{\theta}^{(n+1)} = \arg \max_{\boldsymbol{\theta}} F[q^{(n+1)}, \boldsymbol{\theta}]$.

In order to implement the EM algorithm, we need to be able to do the above maximizations in practice. Fortunately, it can be shown (see, e.g., Neal and Hinton, 1999) that the result of the maximization at the E-step is

$$q^{(n+1)}(\mathbf{x}_{0:T}) = p(\mathbf{x}_{0:T} \mid \mathbf{y}_{1:T}, \boldsymbol{\theta}^{(n)}). \quad (16.25)$$

Plugging this into the expression of $F[q^{(n+1)}(\mathbf{x}_{0:T}), \boldsymbol{\theta}]$ gives

$$\begin{aligned} & F[q^{(n+1)}(\mathbf{x}_{0:T}), \boldsymbol{\theta}] \\ &= \int p(\mathbf{x}_{0:T} \mid \mathbf{y}_{1:T}, \boldsymbol{\theta}^{(n)}) \log p(\mathbf{x}_{0:T}, \mathbf{y}_{1:T} \mid \boldsymbol{\theta}) \, d\mathbf{x}_{0:T} \\ & - \int p(\mathbf{x}_{0:T} \mid \mathbf{y}_{1:T}, \boldsymbol{\theta}^{(n)}) \log p(\mathbf{x}_{0:T} \mid \mathbf{y}_{1:T}, \boldsymbol{\theta}^{(n)}) \, d\mathbf{x}_{0:T}. \end{aligned} \quad (16.26)$$

Because the latter term does not depend on $\boldsymbol{\theta}$, maximizing $F[q^{(n+1)}, \boldsymbol{\theta}]$ is equivalent to maximizing the first term above, which in the EM context is commonly denoted as

$$\mathcal{Q}(\boldsymbol{\theta}, \boldsymbol{\theta}^{(n)}) = \int p(\mathbf{x}_{0:T} \mid \mathbf{y}_{1:T}, \boldsymbol{\theta}^{(n)}) \log p(\mathbf{x}_{0:T}, \mathbf{y}_{1:T} \mid \boldsymbol{\theta}) \, d\mathbf{x}_{0:T}, \quad (16.27)$$

which is thus the expectation of the logarithm of the complete data likelihood $p(\mathbf{x}_{0:T}, \mathbf{y}_{1:T} \mid \boldsymbol{\theta})$ over the full joint posterior of the states given the parameters $\boldsymbol{\theta}^{(n)}$. The resulting algorithm is the following.

Algorithm 16.8 (EM algorithm). *The EM algorithm consists of the following steps.*

- Start from an initial guess $\boldsymbol{\theta}^{(0)}$.
- For $n = 0, 1, 2, \dots$ do the following steps:
 1. E-step: compute $\mathcal{Q}(\boldsymbol{\theta}, \boldsymbol{\theta}^{(n)})$.
 2. M-step: compute $\boldsymbol{\theta}^{(n+1)} = \arg \max_{\boldsymbol{\theta}} \mathcal{Q}(\boldsymbol{\theta}, \boldsymbol{\theta}^{(n)})$.

Due to the Markovian structure of the state space model in Equation (16.1), the complete data log-likelihood has the form

$$\begin{aligned} & \log p(\mathbf{x}_{0:T}, \mathbf{y}_{1:T} \mid \boldsymbol{\theta}) \\ &= \log p(\mathbf{x}_0 \mid \boldsymbol{\theta}) + \sum_{k=1}^T \log p(\mathbf{x}_k \mid \mathbf{x}_{k-1}, \boldsymbol{\theta}) + \sum_{k=1}^T \log p(\mathbf{y}_k \mid \mathbf{x}_k, \boldsymbol{\theta}). \end{aligned} \quad (16.28)$$

The expression for $\mathcal{Q}$ in Equation (16.27) and thus the E-step in Algorithm 16.8 now reduces to computation of (see Schön et al., 2011)

$$\mathcal{Q}(\boldsymbol{\theta}, \boldsymbol{\theta}^{(n)}) = I_1(\boldsymbol{\theta}, \boldsymbol{\theta}^{(n)}) + I_2(\boldsymbol{\theta}, \boldsymbol{\theta}^{(n)}) + I_3(\boldsymbol{\theta}, \boldsymbol{\theta}^{(n)}), \quad (16.29)$$

where

$$\begin{aligned}
 I_1(\boldsymbol{\theta}, \boldsymbol{\theta}^{(n)}) &= \int p(\mathbf{x}_0 \mid \mathbf{y}_{1:T}, \boldsymbol{\theta}^{(n)}) \log p(\mathbf{x}_0 \mid \boldsymbol{\theta}) \, d\mathbf{x}_0, \\
 I_2(\boldsymbol{\theta}, \boldsymbol{\theta}^{(n)}) &= \sum_{k=1}^T \int p(\mathbf{x}_k, \mathbf{x}_{k-1} \mid \mathbf{y}_{1:T}, \boldsymbol{\theta}^{(n)}) \\
 &\quad \times \log p(\mathbf{x}_k \mid \mathbf{x}_{k-1}, \boldsymbol{\theta}) \, d\mathbf{x}_k \, d\mathbf{x}_{k-1}, \\
 I_3(\boldsymbol{\theta}, \boldsymbol{\theta}^{(n)}) &= \sum_{k=1}^T \int p(\mathbf{x}_k \mid \mathbf{y}_{1:T}, \boldsymbol{\theta}^{(n)}) \log p(\mathbf{y}_k \mid \mathbf{x}_k, \boldsymbol{\theta}) \, d\mathbf{x}_k.
 \end{aligned} \tag{16.30}$$

The above expectations are over the smoothing distribution, and the key thing to observe is that we do not need to compute expectations over the full posterior, but only over the smoothing distributions $p(\mathbf{x}_k \mid \mathbf{y}_{1:T}, \boldsymbol{\theta}^{(n)})$ and pairwise smoothing distributions $p(\mathbf{x}_k, \mathbf{x}_{k-1} \mid \mathbf{y}_{1:T}, \boldsymbol{\theta}^{(n)})$. It turns out that the required expectations can be easily (approximately) evaluated using smoother results. In the case of linear state space models, we can find a closed form expression for the above integrals in terms of RTS smoother results. In the non-linear case, we can approximate the integrals by using non-linear smoothers such as Gaussian smoothers. In the more general probabilistic state space model, we can use particle smoothers to approximate them.

On the E-step of Algorithm 16.8, we need to maximize the expression for $\mathcal{Q}$ in Equation (16.29) with respect to the parameters $\boldsymbol{\theta}$. In principle, we can utilize various gradient-free and gradient-based optimization methods (see, e.g., Luenberger and Ye, 2008) for doing that, but the most useful case occurs when we can do the maximization analytically via setting the gradient to zero:

$$\frac{\partial \mathcal{Q}(\boldsymbol{\theta}, \boldsymbol{\theta}^{(n)})}{\partial \boldsymbol{\theta}} = 0. \tag{16.31}$$

This happens, for example, when estimating the parameters of linear state space models and in certain classes of non-linear state space models.

It turns out that we can calculate MAP estimates using the EM algorithm by replacing $p(\mathbf{x}_{0:T}, \mathbf{y}_{1:T} \mid \boldsymbol{\theta})$ in Equation (16.27) with $p(\mathbf{x}_{0:T}, \mathbf{y}_{1:T}, \boldsymbol{\theta})$. In practice, this can be implemented by maximizing $\mathcal{Q}(\boldsymbol{\theta}, \boldsymbol{\theta}^{(n)}) + \log p(\boldsymbol{\theta})$ at the M-step instead of the plain $\mathcal{Q}$.

As a side product of the EM formulation above, we also get a method to compute the gradient of the energy function needed in gradient-based optimization for finding the MAP or ML estimates. *Fisher's identity* (see,

e.g., Cappé et al., 2005) states that if we evaluate the gradient of $\mathcal{Q}$ at $\theta^{(n)} = \theta$, we get exactly the gradient of the marginal log-likelihood. This implies that the gradient of the energy function can be evaluated as

$$\frac{\partial \varphi_T(\theta)}{\partial \theta} = -\frac{\partial \log p(\theta)}{\partial \theta} - \frac{\partial \mathcal{Q}(\theta, \theta^{(n)})}{\partial \theta} \Big|_{\theta^{(n)} = \theta}. \quad (16.32)$$

Note that here we refer to the above identity as Fisher's identity although the original identity is the relationship with the log marginal likelihood and $\mathcal{Q}$, not with the log posterior and $\mathcal{Q}$. In any case this identity is useful in linear state space models, because it is often easier to compute and computationally lighter (Segal and Weinstein, 1989; Olsson et al., 2007). However, in non-linear state space models it is not as useful, because the approximations involved in computation of the filtering and smoothing solutions often cause the gradient to have different approximations from the energy function approximation implied by the same method. That is, the gradient approximation computed with Fisher's identity and Gaussian smoothing might not exactly match the gradient of the energy function approximation computed with the corresponding Gaussian filter. However, in the case of particle filters, Fisher's identity provides a feasible way to approximate the gradient of the energy function.

16.3 Practical Parameter Estimation in State Space Models

In this section we discuss practical parameter estimation methods for state space models using linear Kalman filters and RTS smoothers, Gaussian approximation-based non-linear Kalman filters and RTS smoothers, and particle filters and smoothers. But before going to them, we outline the simple but sometimes effective state augmentation approach.

16.3.1 State Augmentation Approach

Before going to more elaborate parameter estimation methods for state space models, we recall that already in Chapter 3 we used the Kalman filter for estimating static parameters in a regression model. The same approach can be generalized to the *state augmentation approach*, which simply means that we augment the parameter as part of the state. For example, let us say that we have a non-linear model with unknown parameters θ :

$$\begin{aligned} \mathbf{x}_k &= \mathbf{f}(\mathbf{x}_{k-1}, \theta) + \mathbf{q}_{k-1}, \\ \mathbf{y}_k &= \mathbf{h}(\mathbf{x}_k, \theta) + \mathbf{r}_k. \end{aligned} \quad (16.33)$$

We can now rewrite the model as

$$\begin{aligned}\boldsymbol{\theta}_k &= \boldsymbol{\theta}_{k-1}, \\ \mathbf{x}_k &= \mathbf{f}(\mathbf{x}_{k-1}, \boldsymbol{\theta}_{k-1}) + \mathbf{q}_{k-1}, \\ \mathbf{y}_k &= \mathbf{h}(\mathbf{x}_k, \boldsymbol{\theta}_k) + \mathbf{r}_k,\end{aligned}\tag{16.34}$$

where the dynamic model for the parameter essentially says that it is constant. If we now redefine the state as $\tilde{\mathbf{x}}_k = (\mathbf{x}_k, \boldsymbol{\theta}_k)$, we get a state space model of the form

$$\begin{aligned}\tilde{\mathbf{x}}_k &= \tilde{\mathbf{f}}(\tilde{\mathbf{x}}_{k-1}) + \tilde{\mathbf{q}}_{k-1}, \\ \mathbf{y}_k &= \mathbf{h}(\tilde{\mathbf{x}}_k) + \mathbf{r}_k,\end{aligned}\tag{16.35}$$

which does not contain any unknown parameter anymore. The problem in this *state augmentation* is the singularity of the dynamic model for the parameter. It works well when the whole system is linear and we do not have any approximation errors in the estimator. If the parameters appear linearly in the system, it sometimes is a good idea to include the parameters as part of the state. However, this can also fail sometimes.

With approximate non-linear filters, the singularity of the parameter dynamic model indeed causes problems. With non-linear Kalman filters, the Gaussian approximation tends to become singular, which causes the filter to diverge. As discussed in Section 11.5, particle filters have a problem with small amounts of noise in the dynamic model because this causes sample impoverishment. As the noise in the dynamic model above is exactly zero, this case is particularly problematic for particle filters.

A common way to circumvent the problem is to introduce artificial noise to the dynamic model of the parameter, that is, replace $\boldsymbol{\theta}_k = \boldsymbol{\theta}_{k-1}$ with

$$\boldsymbol{\theta}_k = \boldsymbol{\theta}_{k-1} + \boldsymbol{\varepsilon}_{k-1},\tag{16.36}$$

where $\boldsymbol{\varepsilon}_{k-1}$ is a “small” noise process. But the problem is that we are no longer solving the original parameter estimation problem but another one with a time-varying parameter. Anyway, this approach is sometimes applicable and should be considered before jumping into more complicated parameter estimation methods.

There is also a form of Rao–Blackwellization that sometimes helps. This approach is discussed in Section 16.3.6, and the idea is to use a closed form solution for the static parameter (“Rao–Blackwellize” the parameter) and sample only the original state part. This works if the parameter appears in the model in a suitable conjugate form.

16.3.2 Parameter Estimation in Linear State Space Models

Consider the following linear Gaussian state space model with unknown parameters θ :

$$\begin{aligned}\mathbf{x}_k &= \mathbf{A}(\theta) \mathbf{x}_{k-1} + \mathbf{q}_{k-1}, \\ \mathbf{y}_k &= \mathbf{H}(\theta) \mathbf{x}_k + \mathbf{r}_k,\end{aligned}\tag{16.37}$$

where $\mathbf{q}_{k-1} \sim N(\mathbf{0}, \mathbf{Q}(\theta))$, $\mathbf{r}_k \sim N(\mathbf{0}, \mathbf{R}(\theta))$, and $\mathbf{x}_0 \sim N(\mathbf{m}_0(\theta), \mathbf{P}_0(\theta))$. In the above model, for notational convenience, we have assumed that the model matrices do not explicitly depend on time. The energy function and thus the marginal parameter posterior for the linear Gaussian model above can be obtained as follows.

Theorem 16.9 (Energy function for linear Gaussian model). *The recursion for the energy function is given as*

$$\varphi_k(\theta) = \varphi_{k-1}(\theta) + \frac{1}{2} \log |2\pi \mathbf{S}_k(\theta)| + \frac{1}{2} \mathbf{v}_k^\top(\theta) \mathbf{S}_k^{-1}(\theta) \mathbf{v}_k(\theta), \tag{16.38}$$

where the terms $\mathbf{v}_k(\theta)$ and $\mathbf{S}_k(\theta)$ are given by the Kalman filter with the parameters fixed to θ .

- Prediction:

$$\begin{aligned}\mathbf{m}_k^-(\theta) &= \mathbf{A}(\theta) \mathbf{m}_{k-1}(\theta), \\ \mathbf{P}_k^-(\theta) &= \mathbf{A}(\theta) \mathbf{P}_{k-1}(\theta) \mathbf{A}^\top(\theta) + \mathbf{Q}(\theta).\end{aligned}\tag{16.39}$$

- Update:

$$\begin{aligned}\mathbf{v}_k(\theta) &= \mathbf{y}_k - \mathbf{H}(\theta) \mathbf{m}_k^-(\theta), \\ \mathbf{S}_k(\theta) &= \mathbf{H}(\theta) \mathbf{P}_k^-(\theta) \mathbf{H}^\top(\theta) + \mathbf{R}(\theta), \\ \mathbf{K}_k(\theta) &= \mathbf{P}_k^-(\theta) \mathbf{H}^\top(\theta) \mathbf{S}_k^{-1}(\theta), \\ \mathbf{m}_k(\theta) &= \mathbf{m}_k^-(\theta) + \mathbf{K}_k(\theta) \mathbf{v}_k(\theta), \\ \mathbf{P}_k(\theta) &= \mathbf{P}_k^-(\theta) - \mathbf{K}_k(\theta) \mathbf{S}_k(\theta) \mathbf{K}_k^\top(\theta).\end{aligned}\tag{16.40}$$

Proof The Kalman filter gives us the Gaussian predictive distribution $p(\mathbf{x}_k \mid \mathbf{y}_{1:k-1}, \theta) = N(\mathbf{x}_k \mid \mathbf{m}_k^-(\theta), \mathbf{P}_k^-(\theta))$, which via Theorem 16.1 thus gives

$$\begin{aligned}p(\mathbf{y}_k \mid \mathbf{y}_{1:k-1}, \theta) &= \int N(\mathbf{y}_k \mid \mathbf{H}(\theta) \mathbf{x}_k, \mathbf{R}(\theta)) N(\mathbf{x}_k \mid \mathbf{m}_k^-(\theta), \mathbf{P}_k^-(\theta)) d\mathbf{x}_k \\ &= N(\mathbf{y}_k \mid \mathbf{H}(\theta) \mathbf{m}_k^-(\theta), \mathbf{H}(\theta) \mathbf{P}_k^-(\theta) \mathbf{H}^\top(\theta) + \mathbf{R}(\theta)).\end{aligned}\tag{16.41}$$

The rest follows from Theorem 16.4. □

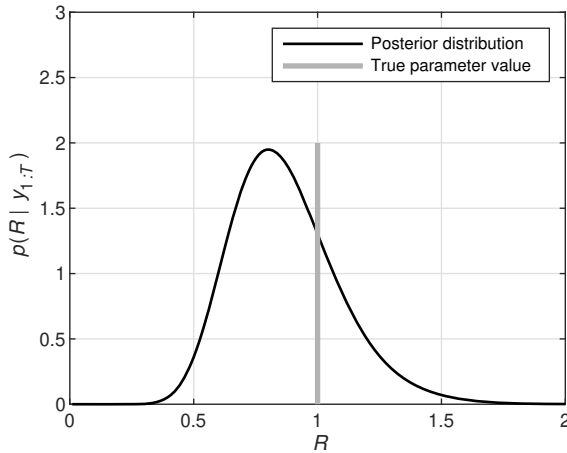


Figure 16.1 Posterior distribution of noise variance R in the Gaussian random walk model (see Example 6.4).

Thus if we fix θ and run the above algorithm from $\varphi_0(\theta) = -\log p(\theta)$ at $k = 0$ to the step $k = T$, then the full energy function is $\varphi_T(\theta)$. That is, the marginal posterior density at the point θ can be evaluated up to a normalization constant by Equation (16.10) as

$$p(\theta \mid \mathbf{y}_{1:T}) \propto \exp(-\varphi_T(\theta)).$$

Given the energy function, it is now easy to implement, for example, a Metropolis–Hastings-based MCMC sampler for generating a Monte Carlo approximation of the posterior distribution or to use the energy function in a gradient-free optimization for finding the ML or MAP estimates of the parameters.

Example 16.10 (Parameter posterior for Gaussian random walk). *The posterior distribution of the noise variance $p(R \mid \mathbf{y}_{1:T})$ for the Gaussian random walk model in Example 6.4 is shown in Figure 16.1. A formally uniform prior $p(R) \propto 1$ was assumed. The true value used in the simulation is indeed well within the high density area of the posterior distribution. However, it can also be seen that if we computed the MAP (or equivalently ML) estimate of the parameter, we would get a smaller value than the true one.*

In order to implement a gradient-based optimization method, we need to have the gradient of the energy function as well. One way to implement

the gradient computation is by first differentiating the energy function expression in Theorem 16.9 term-by-term and then each of the Kalman filter equations. This results in a recursion called *sensitivity equations* (Gupta and Mehra, 1974; Cappé et al., 2005), which can be evaluated along with the Kalman filtering computations. The equations are given in Theorem A.10 in Section A.10. Another way to compute the gradient is by using Fisher's identity (16.32), but before going into that, let us take a look at the EM algorithm for linear Gaussian models.

Recall that for implementing the EM algorithm we need to be able to compute the expectations in Equations (16.30), which in terms requires the knowledge of the smoothing distributions and pairwise smoothing distributions. Fortunately, by Equations (12.5) and (12.12) we know that

$$\begin{aligned} p(\mathbf{x}_k \mid \mathbf{y}_{1:T}, \boldsymbol{\theta}^{(n)}) &= N(\mathbf{x}_k \mid \mathbf{m}_k^s, \mathbf{P}_k^s), \\ p(\mathbf{x}_k, \mathbf{x}_{k-1} \mid \mathbf{y}_{1:T}, \boldsymbol{\theta}^{(n)}) &= N\left(\begin{pmatrix} \mathbf{x}_k \\ \mathbf{x}_{k-1} \end{pmatrix} \mid \begin{pmatrix} \mathbf{m}_k^s \\ \mathbf{m}_{k-1}^s \end{pmatrix}, \begin{pmatrix} \mathbf{P}_k^s & \mathbf{P}_k^s \mathbf{G}_{k-1}^\top \\ \mathbf{G}_{k-1} \mathbf{P}_k^s & \mathbf{P}_{k-1}^s \end{pmatrix}\right), \end{aligned} \quad (16.42)$$

where the means, covariances, and gains are computed with an RTS smoother with the model parameters fixed to $\boldsymbol{\theta}^{(n)}$. Note that in the EM algorithms appearing in the literature, the cross term $\mathbf{P}_k^s \mathbf{G}_{k-1}^\top$ is sometimes computed with a separate recursion (see, e.g., Shumway and Stoffer, 1982), but in fact this is unnecessary due to the above. The required expectations for EM in Equations (16.30) can now be computed in closed form, and the result is the following (cf. Shumway and Stoffer, 1982).

Theorem 16.11 (Evaluation of $\mathcal{Q}$ for linear Gaussian model). *The expression for $\mathcal{Q}$ for the linear Gaussian model in Equation (16.37) can be written as*

$$\begin{aligned} \mathcal{Q}(\boldsymbol{\theta}, \boldsymbol{\theta}^{(n)}) &= -\frac{1}{2} \log |2\pi \mathbf{P}_0(\boldsymbol{\theta})| - \frac{T}{2} \log |2\pi \mathbf{Q}(\boldsymbol{\theta})| - \frac{T}{2} \log |2\pi \mathbf{R}(\boldsymbol{\theta})| \\ &\quad - \frac{1}{2} \operatorname{tr} \left\{ \mathbf{P}_0^{-1}(\boldsymbol{\theta}) \left[\mathbf{P}_0^s + (\mathbf{m}_0^s - \mathbf{m}_0(\boldsymbol{\theta})) (\mathbf{m}_0^s - \mathbf{m}_0(\boldsymbol{\theta}))^\top \right] \right\} \\ &\quad - \frac{T}{2} \operatorname{tr} \left\{ \mathbf{Q}^{-1}(\boldsymbol{\theta}) \left[\boldsymbol{\Sigma} - \mathbf{C} \mathbf{A}^\top(\boldsymbol{\theta}) - \mathbf{A}(\boldsymbol{\theta}) \mathbf{C}^\top + \mathbf{A}(\boldsymbol{\theta}) \boldsymbol{\Phi} \mathbf{A}^\top(\boldsymbol{\theta}) \right] \right\} \\ &\quad - \frac{T}{2} \operatorname{tr} \left\{ \mathbf{R}^{-1}(\boldsymbol{\theta}) \left[\mathbf{D} - \mathbf{B} \mathbf{H}^\top(\boldsymbol{\theta}) - \mathbf{H}(\boldsymbol{\theta}) \mathbf{B}^\top + \mathbf{H}(\boldsymbol{\theta}) \boldsymbol{\Sigma} \mathbf{H}^\top(\boldsymbol{\theta}) \right] \right\}, \end{aligned} \quad (16.43)$$

where the following quantities are computed from the results of RTS smoothers run with parameter values $\boldsymbol{\theta}^{(n)}$:

$$\begin{aligned} \boldsymbol{\Sigma} &= \frac{1}{T} \sum_{k=1}^T \mathbf{P}_k^s + \mathbf{m}_k^s [\mathbf{m}_k^s]^\top, \\ \boldsymbol{\Phi} &= \frac{1}{T} \sum_{k=1}^T \mathbf{P}_{k-1}^s + \mathbf{m}_{k-1}^s [\mathbf{m}_{k-1}^s]^\top, \\ \mathbf{B} &= \frac{1}{T} \sum_{k=1}^T \mathbf{y}_k [\mathbf{m}_k^s]^\top, \\ \mathbf{C} &= \frac{1}{T} \sum_{k=1}^T \mathbf{P}_k^s \mathbf{G}_{k-1}^\top + \mathbf{m}_k^s [\mathbf{m}_{k-1}^s]^\top, \\ \mathbf{D} &= \frac{1}{T} \sum_{k=1}^T \mathbf{y}_k \mathbf{y}_k^\top. \end{aligned} \quad (16.44)$$

The usefulness of the EM algorithm for linear state space models stems from the fact that if the parameters are selected to be some of the full model matrices (or initial mean), we can indeed perform the M-step of the EM algorithm in closed form. The same thing happens if the parameters appear linearly in one of the model matrices (e.g., are some subcomponents of the matrices), but application of the EM algorithm to the estimation of the full matrices is the classical result. By setting the gradients of $\partial \mathcal{Q}(\boldsymbol{\theta}, \boldsymbol{\theta}^{(n)}) / \partial \boldsymbol{\theta}$

to zero for each $\theta = \{\mathbf{A}, \mathbf{H}, \mathbf{Q}, \mathbf{R}, \mathbf{P}_0, \mathbf{m}_0\}$ separately, we get the following result.

Theorem 16.12 (Maximization of $\mathcal{Q}$ for linear model parameters). *The maximum $\theta^* = \arg \max_{\theta} \mathcal{Q}(\theta, \theta^{(n)})$, when the parameters are selected to be one of the model parameters $\theta \in \{\mathbf{A}, \mathbf{H}, \mathbf{Q}, \mathbf{R}, \mathbf{P}_0, \mathbf{m}_0\}$, can be computed as follows.*

- When $\theta = \mathbf{P}_0$ we get

$$\mathbf{P}_0^* = \mathbf{P}_0^s + (\mathbf{m}_0^s - \mathbf{m}_0)(\mathbf{m}_0^s - \mathbf{m}_0)^\top. \quad (16.45)$$

- When $\theta = \mathbf{A}$ we get

$$\mathbf{A}^* = \mathbf{C} \Phi^{-1}. \quad (16.46)$$

- When $\theta = \mathbf{Q}$ we get

$$\mathbf{Q}^* = \Sigma - \mathbf{C} \mathbf{A}^\top - \mathbf{A} \mathbf{C}^\top + \mathbf{A} \Phi \mathbf{A}^\top. \quad (16.47)$$

- When $\theta = \mathbf{H}$ we get

$$\mathbf{H}^* = \mathbf{B} \Sigma^{-1}. \quad (16.48)$$

- When $\theta = \mathbf{R}$ we get

$$\mathbf{R}^* = \mathbf{D} - \mathbf{H} \mathbf{B}^\top - \mathbf{B} \mathbf{H}^\top + \mathbf{H} \Sigma \mathbf{H}^\top. \quad (16.49)$$

- Finally, the maximum with respect to the initial mean $\theta = \mathbf{m}_0$ is

$$\mathbf{m}_0^* = \mathbf{m}_0^s. \quad (16.50)$$

Obviously the above theorem can also be used for solving the maximum of $\mathcal{Q}$ with respect to any subset of model matrices by solving the resulting equations jointly. The EM algorithm for finding the maximum likelihood estimates of the linear state space model parameters is now the following.

Algorithm 16.13 (EM algorithm for linear state space models). *Let θ contain some subset of the model parameters $\{\mathbf{A}, \mathbf{H}, \mathbf{Q}, \mathbf{R}, \mathbf{P}_0, \mathbf{m}_0\}$. We can find maximum likelihood estimates of them via the following iteration.*

- Start from some initial guess $\theta^{(0)}$.
- For $n = 0, 1, 2, \dots$ do the following steps.
 1. E-step: Run the RTS smoother using the current parameter values in $\theta^{(n)}$, and compute the quantities in Equation (16.44) from the smoother results.
 2. M-step: Find new parameter values by using Equations (16.45) – (16.50), and store them in $\theta^{(n+1)}$.

The expression for $\mathcal{Q}(\boldsymbol{\theta}, \boldsymbol{\theta}^{(n)})$ also provides an “easy gradient recipe” (Olsson et al., 2007) for computation of the energy function gradient via Fisher’s identity (Equation (16.32)), as it enables the computation of the gradient without an additional recursion (sensitivity equations). The resulting expression is given in Theorem A.11 in Section A.10.

16.3.3 Parameter Estimation with Gaussian Filtering and Smoothing

One way to implement parameter estimation in non-linear models is by replacing the Kalman filters and RTS smoothers used in the linear case with their non-linear counterparts. Let’s consider parameter estimation in models of the form

$$\begin{aligned}\mathbf{x}_k &= \mathbf{f}(\mathbf{x}_{k-1}, \boldsymbol{\theta}) + \mathbf{q}_{k-1}, \\ \mathbf{y}_k &= \mathbf{h}(\mathbf{x}_k, \boldsymbol{\theta}) + \mathbf{r}_k,\end{aligned}\tag{16.51}$$

where $\mathbf{q}_{k-1} \sim N(\mathbf{0}, \mathbf{Q}(\boldsymbol{\theta}))$, $\mathbf{r}_k \sim N(\mathbf{0}, \mathbf{R}(\boldsymbol{\theta}))$, and $\mathbf{x}_0 \sim N(\mathbf{m}_0(\boldsymbol{\theta}), \mathbf{P}_0(\boldsymbol{\theta}))$. The energy function can now be approximated with the following Gaussian filtering-based algorithm.

Algorithm 16.14 (Gaussian filtering-based energy function). *The recursion for the approximate energy function is*

$$\varphi_k(\boldsymbol{\theta}) \simeq \varphi_{k-1}(\boldsymbol{\theta}) + \frac{1}{2} \log |2\pi \mathbf{S}_k(\boldsymbol{\theta})| + \frac{1}{2} \mathbf{v}_k^\top(\boldsymbol{\theta}) \mathbf{S}_k^{-1}(\boldsymbol{\theta}) \mathbf{v}_k(\boldsymbol{\theta}), \tag{16.52}$$

where the terms $\mathbf{v}_k(\boldsymbol{\theta})$ and $\mathbf{S}_k(\boldsymbol{\theta})$ are given by the Gaussian filter with the parameters fixed to $\boldsymbol{\theta}$.

- *Prediction:*

$$\begin{aligned}\mathbf{m}_k^-(\boldsymbol{\theta}) &= \int \mathbf{f}(\mathbf{x}_{k-1}, \boldsymbol{\theta}) N(\mathbf{x}_{k-1} \mid \mathbf{m}_{k-1}(\boldsymbol{\theta}), \mathbf{P}_{k-1}(\boldsymbol{\theta})) d\mathbf{x}_{k-1}, \\ \mathbf{P}_k^-(\boldsymbol{\theta}) &= \int (\mathbf{f}(\mathbf{x}_{k-1}, \boldsymbol{\theta}) - \mathbf{m}_k^-(\boldsymbol{\theta})) (\mathbf{f}(\mathbf{x}_{k-1}, \boldsymbol{\theta}) - \mathbf{m}_k^-(\boldsymbol{\theta}))^\top \\ &\quad \times N(\mathbf{x}_{k-1} \mid \mathbf{m}_{k-1}(\boldsymbol{\theta}), \mathbf{P}_{k-1}(\boldsymbol{\theta})) d\mathbf{x}_{k-1} + \mathbf{Q}_{k-1}(\boldsymbol{\theta}).\end{aligned}\tag{16.53}$$

- *Update:*

$$\begin{aligned}
 \mu_k(\theta) &= \int \mathbf{h}(\mathbf{x}_k, \theta) N(\mathbf{x}_k \mid \mathbf{m}_k^-(\theta), \mathbf{P}_k^-(\theta)) d\mathbf{x}_k, \\
 \mathbf{v}_k(\theta) &= \mathbf{y}_k - \mu_k(\theta), \\
 \mathbf{S}_k(\theta) &= \int (\mathbf{h}(\mathbf{x}_k, \theta) - \mu_k(\theta)) (\mathbf{h}(\mathbf{x}_k, \theta) - \mu_k(\theta))^\top \\
 &\quad \times N(\mathbf{x}_k \mid \mathbf{m}_k^-(\theta), \mathbf{P}_k^-(\theta)) d\mathbf{x}_k + \mathbf{R}_k(\theta), \\
 \mathbf{C}_k(\theta) &= \int (\mathbf{x}_k - \mathbf{m}_k^-(\theta)) (\mathbf{h}(\mathbf{x}_k, \theta) - \mu_k(\theta))^\top \\
 &\quad \times N(\mathbf{x}_k \mid \mathbf{m}_k^-(\theta), \mathbf{P}_k^-(\theta)) d\mathbf{x}_k, \\
 \mathbf{K}_k(\theta) &= \mathbf{C}_k(\theta) \mathbf{S}_k^{-1}(\theta), \\
 \mathbf{m}_k(\theta) &= \mathbf{m}_k^-(\theta) + \mathbf{K}_k(\theta) \mathbf{v}_k(\theta), \\
 \mathbf{P}_k(\theta) &= \mathbf{P}_k^-(\theta) - \mathbf{K}_k(\theta) \mathbf{S}_k(\theta) \mathbf{K}_k^\top(\theta).
 \end{aligned} \tag{16.54}$$

Derivation This approximation can be derived in the same way as the linear case in Theorem 16.9 except that Gaussian moment matching-based approximations are used instead of the true Gaussian distributions. $\square$

The above energy function can now be directly used in MCMC sampling or in gradient-free optimization algorithms for computing ML or MAP estimates. However, because the energy function is based on a Gaussian approximation, the implied posterior distribution is an approximation as well, and thus the parameter estimates will be biased. The posterior distribution approximation is also typically thinner than the true posterior distribution, and thus the uncertainty in the parameter is underestimated. This issue is illustrated in Example 16.17.

It is also possible to compute the derivatives of the above energy function analogously to the linear case. In the case of the extended Kalman filter (EKF), the derivatives can be easily derived by formally differentiating the EKF equations (see Mbalawata et al., 2013). When sigma point filters are used, a similar approach works, but additional care is needed for computation of the derivative of the square root matrix $\partial\sqrt{\mathbf{P}(\theta)}/\partial\theta_i$ arising in the equations. The equations for computing the derivatives of the energy function are given in Algorithm A.12.

To compute the expectations required for implementing the EM algorithm, we can approximate the integrals in Equations (16.30) using the Gaussian assumed density approximation (i.e., moment matching). The resulting expression for $\mathcal{Q}$ is the following.

Algorithm 16.15 (Evaluation of $\mathcal{Q}$ via Gaussian smoothing). *The expression for $\mathcal{Q}$ for the non-linear state space model in Equation (16.51) can be written as*

$$\begin{aligned} \mathcal{Q}(\boldsymbol{\theta}, \boldsymbol{\theta}^{(n)}) & \simeq -\frac{1}{2} \log |2\pi \mathbf{P}_0(\boldsymbol{\theta})| - \frac{T}{2} \log |2\pi \mathbf{Q}(\boldsymbol{\theta})| - \frac{T}{2} \log |2\pi \mathbf{R}(\boldsymbol{\theta})| \\ & - \frac{1}{2} \operatorname{tr} \left\{ \mathbf{P}_0^{-1}(\boldsymbol{\theta}) \left[\mathbf{P}_0^s + (\mathbf{m}_0^s - \mathbf{m}_0(\boldsymbol{\theta})) (\mathbf{m}_0^s - \mathbf{m}_0(\boldsymbol{\theta}))^\top \right] \right\} \\ & - \frac{1}{2} \sum_{k=1}^T \operatorname{tr} \left\{ \mathbf{Q}^{-1}(\boldsymbol{\theta}) \operatorname{E} [(\mathbf{x}_k - \mathbf{f}(\mathbf{x}_{k-1}, \boldsymbol{\theta})) (\mathbf{x}_k - \mathbf{f}(\mathbf{x}_{k-1}, \boldsymbol{\theta}))^\top \mid \mathbf{y}_{1:T}] \right\} \\ & - \frac{1}{2} \sum_{k=1}^T \operatorname{tr} \left\{ \mathbf{R}^{-1}(\boldsymbol{\theta}) \operatorname{E} [(\mathbf{y}_k - \mathbf{h}(\mathbf{x}_k, \boldsymbol{\theta})) (\mathbf{y}_k - \mathbf{h}(\mathbf{x}_k, \boldsymbol{\theta}))^\top \mid \mathbf{y}_{1:T}] \right\}, \end{aligned} \quad (16.55)$$

where the expectations are over the counterparts of the distributions in Equations (16.42) obtained from the Gaussian smoother.

In practice, we can approximate the Gaussian smoother and Gaussian integrals above with Taylor series approximations (EKF/ERTSS) or by sigma point methods such as Gauss–Hermite or spherical cubature integration or the unscented transform. The M-step for the noise parameters can indeed be implemented analogously to the linear case in Theorem 16.12, because the maxima of the above $\mathcal{Q}$ with respect to the noise covariance are simply

$$\begin{aligned} \mathbf{Q}^* &= \frac{1}{T} \sum_{k=1}^T \operatorname{E} [(\mathbf{x}_k - \mathbf{f}(\mathbf{x}_{k-1}, \boldsymbol{\theta})) (\mathbf{x}_k - \mathbf{f}(\mathbf{x}_{k-1}, \boldsymbol{\theta}))^\top \mid \mathbf{y}_{1:T}], \\ \mathbf{R}^* &= \frac{1}{T} \sum_{k=1}^T \operatorname{E} [(\mathbf{y}_k - \mathbf{h}(\mathbf{x}_k, \boldsymbol{\theta})) (\mathbf{y}_k - \mathbf{h}(\mathbf{x}_k, \boldsymbol{\theta}))^\top \mid \mathbf{y}_{1:T}]. \end{aligned} \quad (16.56)$$

The details of implementation of the M-step for other kinds of parameter depend on the actual functional form of $\mathbf{f}$ and $\mathbf{h}$. If the parameters appear linearly in the functions, it is possible to find closed form solutions for the maxima analogously to the linear case (Theorem 16.12). Obviously, even when analytical solutions cannot be found, it would be possible to use iterative optimization methods inside EM. But if iterative methods need to be used anyway, then with the same effort we can try to find the minimum of the energy function directly (recall that this is what EM tries to find as well).

Also in the non-linear case, Fisher's identity (Equation (16.32)), in principle, gives an easy way to evaluate the gradients of the energy function. The problem is that both the energy function and the gradient given by Fisher's identity are approximations, and there is no guarantee that the approximations involved are the same. That is, the derivative given by Fisher's identity might not be exactly the derivative of the approximate energy function given by the Gaussian filter. The derivation of the Fisher's identity-based derivative expression is left as an exercise to the reader (Exercise 16.6).

16.3.4 Parameter Estimation with Iterated Filtering and Smoothing

In Chapters 7, 10, 13, and 14 we saw a collection of iterated filters and smoothers that improve the filter and smoother estimates by using iteration either on the update step or over the whole linearization trajectory. In principle, iterated filters can be used to approximate the energy function in the same way as Gaussian filters simply by taking the final result of iteration as the final filter approximation. We can also use iterated smoothers to find the best linearization trajectory along with the corresponding linearization and then use the energy function provided by the (affine) Kalman filter for parameter estimation, or alternatively, use the corresponding $\mathcal{Q}$ approximation.

Indeed, the aforementioned approaches work well for parameter estimation in principle, but there is one catch: The linearization resulting from the iteration will depend on the model parameters, and hence the iterated linearization result will be a complicated function of the parameters. For evaluation of the energy function this is not a problem, but it causes problems when computing the energy gradient. The gradient of the approximation will no longer be given by, for example, the sensitivity equations, but instead it will also depend on the derivative of the iterated linear linearization result with respect to the parameters. This issue is addressed, for example, in Christianson (1994).

16.3.5 Parameter Estimation via Particle Filtering and Smoothing

Particle filtering can also be used for approximate evaluation of the marginal likelihood and also of the energy function needed in parameter estimation. In the particle filtering approach, we can consider generic

models of the form

$$\begin{aligned}\boldsymbol{\theta} &\sim p(\boldsymbol{\theta}), \\ \mathbf{x}_0 &\sim p(\mathbf{x}_0 \mid \boldsymbol{\theta}), \\ \mathbf{x}_k &\sim p(\mathbf{x}_k \mid \mathbf{x}_{k-1}, \boldsymbol{\theta}), \\ \mathbf{y}_k &\sim p(\mathbf{y}_k \mid \mathbf{x}_k, \boldsymbol{\theta}),\end{aligned}\tag{16.57}$$

where $\boldsymbol{\theta} \in \mathbb{R}^d$ is the unknown parameter to be estimated. The approximate evaluation of the marginal likelihood can be done with the following modification of the SIR particle filter (see, e.g., Andrieu et al., 2004; Creal, 2012).

Algorithm 16.16 (SIR-based energy function approximation). *An approximation to the marginal likelihood of the parameters can be evaluated during the sequential importance resampling (SIR) algorithm (particle filter), as follows.*

- Draw N samples $\mathbf{x}_0^{(i)}$ from the prior:

$$\mathbf{x}_0^{(i)} \sim p(\mathbf{x}_0 \mid \boldsymbol{\theta}), \quad i = 1, \dots, N, \tag{16.58}$$

and set $w_0^{(i)} = 1/N$ for all $i = 1, \dots, N$.

- For each $k = 1, \dots, T$, do the following.

1. Draw samples $\mathbf{x}_k^{(i)}$ from the importance distributions:

$$\mathbf{x}_k^{(i)} \sim \pi(\mathbf{x}_k \mid \mathbf{x}_{k-1}^{(i)}, \mathbf{y}_{1:k}, \boldsymbol{\theta}), \quad i = 1, \dots, N. \tag{16.59}$$

2. Compute the following weights:

$$v_k^{(i)} = \frac{p(\mathbf{y}_k \mid \mathbf{x}_k^{(i)}, \boldsymbol{\theta}) p(\mathbf{x}_k^{(i)} \mid \mathbf{x}_{k-1}^{(i)}, \boldsymbol{\theta})}{\pi(\mathbf{x}_k^{(i)} \mid \mathbf{x}_{k-1}^{(i)}, \mathbf{y}_{1:k}, \boldsymbol{\theta})}, \tag{16.60}$$

and compute the estimate of $p(\mathbf{y}_k \mid \mathbf{y}_{1:k-1}, \boldsymbol{\theta})$ as

$$\hat{p}(\mathbf{y}_k \mid \mathbf{y}_{1:k-1}, \boldsymbol{\theta}) = \sum_i w_{k-1}^{(i)} v_k^{(i)}. \tag{16.61}$$

3. Compute the normalized weights as

$$w_k^{(i)} \propto w_{k-1}^{(i)} v_k^{(i)}. \tag{16.62}$$

4. If the effective number of particles (11.31) is too low, perform resampling.

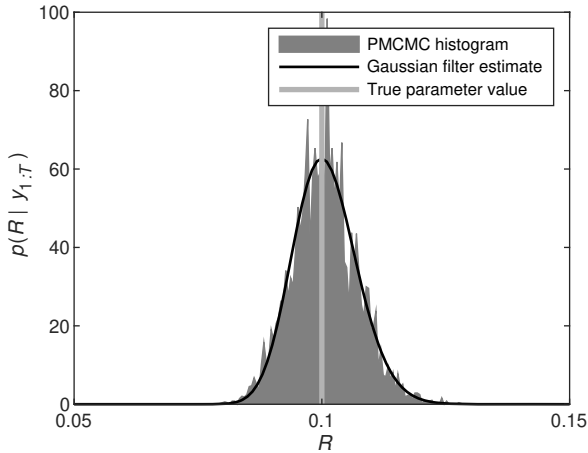


Figure 16.2 Posterior distribution of the noise variance R in the pendulum model (see Example 16.17). The approximation computed with the the Gaussian filter is very close to the reference result computed with the PMCMC.

The approximation of the marginal likelihood of the parameters is

$$p(\mathbf{y}_{1:T} \mid \boldsymbol{\theta}) \approx \prod_k \hat{p}(\mathbf{y}_k \mid \mathbf{y}_{1:k-1}, \boldsymbol{\theta}), \quad (16.63)$$

and the corresponding energy function approximation is

$$\varphi_T(\boldsymbol{\theta}) \approx -\log p(\boldsymbol{\theta}) - \sum_{k=1}^T \log \hat{p}(\mathbf{y}_k \mid \mathbf{y}_{1:k-1}, \boldsymbol{\theta}). \quad (16.64)$$

The energy function approximation could also be computed recursively during the SIR algorithm above.

The particle filter-based energy function approximation can now be used, for example, in the Metropolis–Hastings-based MCMC algorithm. The result is the particle Markov chain Monte Carlo (PMCMC) method (Andrieu et al., 2010) and, more specifically, the particle marginal Metropolis–Hastings (PMMH) algorithm variant of it. It turns out that the resulting MCMC algorithm is exact in the sense that it samples from the right distribution Andrieu et al. (2010). Thus the result is (asymptotically) the same as if we had used the exact energy function instead of the particle filter-based approximation in the MCMC method.

Example 16.17 (Estimation of noise variance in the pendulum model). *Figure 16.2 shows the posterior distribution approximation for the noise variance R computed with a Gaussian filter for the pendulum model in Example 7.6. The figure also shows the histogram of samples from the PMCMC method, which thus should approach the true posterior of the parameter. As can be seen, the posterior distribution estimate of the Gaussian filter (fifth order Gauss–Hermite filter) is a good approximation to the true posterior distribution.*

In principle, it would also be possible to use the likelihood or energy function approximation in gradient-free optimization methods for finding MAP or ML estimates. However, this might turn out to be hard, because even if we fixed the random number generator sequence in the particle filter, the likelihood function would not be continuous in θ (see, e.g., Kantas et al., 2009). This also renders the use of gradient-based optimization methods impossible.

By comparing to the Rao–Blackwellized particle filter in Algorithm 11.13, it is easy to see that the corresponding likelihood approximation can be obtained by setting

$$v_k^{(i)} = \frac{p(\mathbf{y}_k \mid \mathbf{u}_{0:k}^{(i)}, \mathbf{y}_{1:k-1}, \theta) p(\mathbf{u}_k^{(i)} \mid \mathbf{u}_{k-1}^{(i)}, \theta)}{\pi(\mathbf{u}_k^{(i)} \mid \mathbf{u}_{0:k-1}^{(i)}, \mathbf{y}_{1:k}, \theta)}. \quad (16.65)$$

The likelihood approximation itself remains the same as in Equation (16.61):

$$\hat{p}(\mathbf{y}_k \mid \mathbf{y}_{1:k-1}, \theta) = \sum_i w_{k-1}^{(i)} v_k^{(i)}.$$

We can also implement the EM algorithm using particle smoothing. Recall that to implement the EM algorithm, we need to evaluate $\mathcal{Q}(\theta, \theta^{(n)})$ via Equation (16.29), which in turn requires us to evaluate the expectations appearing in Equation (16.30). The actual form of the approximation depends on the particle smoother that we are using. In the case of the backward-simulation particle smoother, we have the following simple algorithm (Wills et al., 2013).

Algorithm 16.18 (Evaluation of $\mathcal{Q}$ via the backward-simulation particle smoother). *Assume that we have simulated S trajectories $\{\tilde{\mathbf{x}}_{0:T}^{(i)} : i = 1, \dots, S\}$ using the backward-simulation particle smoother in Algorithm 15.2 with parameter values fixed to $\theta^{(n)}$. Then the integrals*

in Equation (16.30) can be approximated as

$$\begin{aligned}
 I_1(\boldsymbol{\theta}, \boldsymbol{\theta}^{(n)}) &\approx \frac{1}{S} \sum_{i=1}^S \log p(\tilde{\mathbf{x}}_0^{(i)} | \boldsymbol{\theta}), \\
 I_2(\boldsymbol{\theta}, \boldsymbol{\theta}^{(n)}) &\approx \sum_{k=0}^{T-1} \frac{1}{S} \sum_{i=1}^S \log p(\tilde{\mathbf{x}}_{k+1}^{(i)} | \tilde{\mathbf{x}}_k^{(i)}, \boldsymbol{\theta}), \\
 I_3(\boldsymbol{\theta}, \boldsymbol{\theta}^{(n)}) &\approx \sum_{k=1}^T \frac{1}{S} \sum_{i=1}^S \log p(\mathbf{y}_k | \tilde{\mathbf{x}}_k^{(i)}, \boldsymbol{\theta}).
 \end{aligned} \tag{16.66}$$

If we are using the reweighting (or marginal) particle smoother in Algorithm 15.7, the corresponding expectations can be approximated as follows (Schön et al., 2011).

Algorithm 16.19 (Evaluation of $\mathcal{Q}$ via the reweighting smoother). Assume that we have the set of particles $\{\mathbf{x}_k^{(i)} : k = 0, \dots, T; i = 1, \dots, N\}$ representing the filtering distribution and we have calculated the weights $\{w_{k|T}^{(i)} : k = 0, \dots, T; i = 1, \dots, N\}$ using Algorithm 15.7. Then we can approximate the integrals in Equation (16.30) as follows:

$$\begin{aligned}
 I_1(\boldsymbol{\theta}, \boldsymbol{\theta}^{(n)}) &\approx \sum_i w_{0|T}^{(i)} \log p(\mathbf{x}_0^{(i)} | \boldsymbol{\theta}), \\
 I_2(\boldsymbol{\theta}, \boldsymbol{\theta}^{(n)}) &\approx \sum_{k=0}^{T-1} \sum_i \sum_j \frac{w_{k+1|T}^{(j)} w_k^{(i)} p(\mathbf{x}_{k+1}^{(j)} | \mathbf{x}_k^{(i)}, \boldsymbol{\theta}^{(n)})}{\left[\sum_l w_k^{(l)} p(\mathbf{x}_{k+1}^{(j)} | \mathbf{x}_k^{(l)}, \boldsymbol{\theta}^{(n)}) \right]} \\
 &\quad \times \log p(\mathbf{x}_{k+1}^{(j)} | \mathbf{x}_k^{(i)}, \boldsymbol{\theta}), \\
 I_3(\boldsymbol{\theta}, \boldsymbol{\theta}^{(n)}) &\approx \sum_{k=1}^T \sum_i w_{k|T}^{(i)} \log p(\mathbf{y}_k | \mathbf{x}_k^{(i)}, \boldsymbol{\theta}).
 \end{aligned} \tag{16.67}$$

By differentiating the expressions in the above algorithms with respect to $\boldsymbol{\theta}$, we can also get an approximation for $\partial \mathcal{Q}(\boldsymbol{\theta}, \boldsymbol{\theta}^{(n)}) / \partial \boldsymbol{\theta}$. This approximation can be further used in Fisher's identity in Equation (16.32) to give approximations for the gradients of the energy function (Ninness et al., 2010). For more information on this kind of approach as well as other methods for particle filtering-based parameter estimation, the reader is referred to Andrieu et al. (2004), Kantas et al. (2009), and Poyiadjis et al. (2011).

16.3.6 Rao–Blackwellization of Parameters

In this section we show how Rao–Blackwellization can sometimes be used for marginalizing out the static parameters in state space models. Let's start by considering the following generalized version of the pendulum example used in Särkkä (2006) and Särkkä and Sottinen (2008):

$$\begin{aligned}\mathbf{x}_k &= \mathbf{f}(\mathbf{x}_{k-1}) + \mathbf{q}_{k-1}, \\ y_k &= h(\mathbf{x}_k) + r_k, \\ r_k &\sim N(0, R), \\ R &\sim \text{Inv-}\chi^2(\nu_0, R_0),\end{aligned}\tag{16.68}$$

where $\mathbf{q}_{k-1} \sim N(\mathbf{0}, \mathbf{Q})$. This is thus the same kind of non-linear state space model that we have already considered in this book, except that here the measurement noise variance R is considered as unknown and given an inverse-chi-squared prior distribution $\text{Inv-}\chi^2(\nu_0, R_0)$.

It turns out that we can do sequential Monte Carlo sampling in this model such that we do not need to sample the values of R . Instead, it is enough to sample the state values and then carry the parameters of the distribution of R , conditional on the previous measurements and the histories of samples. The idea is the following.

1. Assume that we have generated a set of particle histories $\{w_k^{(i)}, \mathbf{x}_{0:k}^{(i)} : i = 1, \dots, N\}$ that approximate the full distribution of the states as follows:

$$p(\mathbf{x}_{0:k-1} \mid y_{1:k-1}) \approx \sum_i w_{k-1}^{(i)} \delta(\mathbf{x}_{0:k-1} - \mathbf{x}_{0:k-1}^{(i)}),\tag{16.69}$$

which is thus the conventional SIR filter approximation when we store the full sample histories (see Section 15.1). Further assume that the conditional distribution of R given measurements $y_{1:k-1}$ and the sampled state history $\mathbf{x}_{0:k-1}^{(i)}$ is

$$p(R \mid \mathbf{x}_{0:k-1}^{(i)}, y_{1:k-1}) = \text{Inv-}\chi^2(R \mid \nu_{k-1}^{(i)}, R_{k-1}^{(i)}),\tag{16.70}$$

where $\nu_{k-1}^{(i)}, R_{k-1}^{(i)}$ have already been computed for each i . Then we have the following approximation for the full distribution of states and parameters:

$$\begin{aligned}p(\mathbf{x}_{0:k-1}, R \mid y_{1:k-1}) \\ &= p(R \mid \mathbf{x}_{0:k-1}, y_{1:k-1}) p(\mathbf{x}_{0:k-1} \mid y_{1:k-1}) \\ &\approx \sum_i w_{k-1}^{(i)} \text{Inv-}\chi^2(R \mid \nu_{k-1}^{(i)}, R_{k-1}^{(i)}) \delta(\mathbf{x}_{0:k-1} - \mathbf{x}_{0:k-1}^{(i)}).\end{aligned}\tag{16.71}$$

2. Let us now draw samples from an importance distribution:

$$\mathbf{x}_k^{(i)} \sim \pi(\mathbf{x}_k^{(i)} | \mathbf{x}_{k-1}^{(i)}, y_k). \quad (16.72)$$

3. We can now evaluate the likelihood of the measurement given $\mathbf{x}_k^{(i)}$ and the previous measurements as follows:

$$\begin{aligned} p(y_k | \mathbf{x}_k^{(i)}, y_{1:k-1}) \\ &= \int N(y_k | h(\mathbf{x}_k^{(i)}), R) \text{Inv-}\chi^2(R | v_{k-1}^{(i)}, R_{k-1}^{(i)}) \, dR \\ &= t_{v_k^{(i)}}(y_k | h(\mathbf{x}_k^{(i)}), R_k^{(i)}), \end{aligned} \quad (16.73)$$

where the parameters of the Student's t -distribution above are

$$\begin{aligned} v_k^{(i)} &= v_{k-1}^{(i)} + 1, \\ R_k^{(i)} &= \frac{v_{k-1}^{(i)} R_{k-1}^{(i)} + (y_k - h(\mathbf{x}_k^{(i)}))^2}{v_{k-1}^{(i)} + 1}. \end{aligned} \quad (16.74)$$

This allows us to compute the next step importance weights for the SIR algorithm as follows:

$$w_k^{(i)} \propto \frac{p(y_k | \mathbf{x}_k^{(i)}, y_{1:k-1}) N(\mathbf{x}_k^{(i)} | \mathbf{f}(\mathbf{x}_{k-1}^{(i)}), \mathbf{Q})}{\pi(\mathbf{x}_k^{(i)} | \mathbf{x}_{k-1}^{(i)}, y_k)}. \quad (16.75)$$

4. Given the measurement and the state, we can further compute the conditional distribution of R given $y_{1:k}$ and $\mathbf{x}_{0:k}^{(i)}$:

$$p(R | \mathbf{x}_{0:k}^{(i)}, y_{1:k}) = \text{Inv-}\chi^2(R | v_k^{(i)}, R_k^{(i)}). \quad (16.76)$$

5. Now we again have a similar representation for the filtering distribution as in step 1, and we can set $k \leftarrow k + 1$ and go back to step 1. But before that we can also do resampling jointly for the state histories $\mathbf{x}_{0:k}^{(i)}$ and the parameters $v_k^{(i)}, R_k^{(i)}$ as in the conventional SIR algorithm.

In the above algorithm, we would not actually need to carry the whole state histories in the filter, but theoretically the parameters of the inverse chi squared distributions are indeed conditioned on the full state histories.

The procedure above is a Rao–Blackwellized particle filter where the static parameter R has been marginalized out and is carried via its sufficient statistics $v_k^{(i)}, R_k^{(i)}$ conditioned on the particle histories $\mathbf{x}_{0:k}^{(i)}$ and measurements. In another example given in Särkkä (2006) and Särkkä and Sottinen

(2008), this procedure is used for marginalizing out the unknown population size in a Poisson measurement model. The same idea can be used in various other types of model.

In an abstract sense the method can be applied to a class of models of the form

$$\begin{aligned}\mathbf{x}_k &\sim p(\mathbf{x}_k \mid \mathbf{x}_{k-1}, \boldsymbol{\theta}), \\ \mathbf{y}_k &\sim p(\mathbf{y}_k \mid \mathbf{x}_k, \boldsymbol{\theta}), \\ \boldsymbol{\theta} &\sim p(\boldsymbol{\theta}),\end{aligned}\tag{16.77}$$

where the vector $\boldsymbol{\theta}$ contains the unknown static parameters. Now if the posterior distribution of the parameters $\boldsymbol{\theta}$ depends only on some sufficient statistics

$$\mathcal{T}_k = \mathcal{T}_k(\mathbf{x}_{1:k}, \mathbf{y}_{1:k}),\tag{16.78}$$

and if the sufficient statistics are easy to update recursively, $\mathcal{T}_k \leftarrow \mathcal{T}_{k-1}$, then sampling of the state and parameters can be performed by recursively computing the sufficient statistics conditionally on the sampled states and the measurements analogously to the example above. The original idea of the method seems to have appeared quite independently in Storvik (2002), Fearnhead (2002), and Djuric and Miguez (2002), and it has been applied to estimation of full noise covariances in state space models by Saha et al. (2010).

A particularly useful special case, which includes the example above, is obtained when the dynamic model is independent of the parameters $\boldsymbol{\theta}$. In this case, if conditionally to the state $\mathbf{x}_k$ the prior $p(\boldsymbol{\theta})$ belongs to the conjugate family of the likelihood $p(\mathbf{y}_k \mid \mathbf{x}_k, \boldsymbol{\theta})$, the static parameters $\boldsymbol{\theta}$ can be marginalized out, and only the states need to be sampled. This idea can be extended to the time-varying case if the dynamic model has a form that keeps the predicted distribution of the parameter within the conjugate family (cf. Särkkä and Nummenmaa, 2009).

When the static parameter appears linearly in the model, we recover a noise-free version of the conditionally Gaussian Rao–Blackwellization considered in Section 11.7 (see Schön and Gustafsson, 2003). The Rao–Blackwellized particle filter can then be seen as a time-varying extension of this method in the conditionally linear Gaussian case.

16.4 Exercises

- 16.1 Implement the EM algorithm for ML estimation of the measurement noise variance in the Gaussian random walk model considered in Examples 6.4, 6.7, and 12.3. Test the algorithm with simulated data.
- 16.2 Implement the algorithm for computing the energy function for the Gaussian random walk model as well as its derivative with respect to the noise variance (via the sensitivity equations given in Section A.10). Generate some simulated data, and use a gradient-based optimization method to find the ML estimate of the parameter.
- 16.3 With the Gaussian random walk model, find the expression for the Fisher's identity-based derivative with respect to the noise parameter. Check numerically that it matches the expression obtained with the sensitivity equations.
- 16.4 Implement a random walk Metropolis-based MCMC method for estimating the noise variance in the Gaussian random walk model. Use the Kalman filter to evaluate the energy function. For simplicity, you can assume that the prior for the parameter has the form $p(R) \propto 1$.
- 16.5 Derive the sensitivity equations for the first order extended Kalman filter.
- 16.6 Derive the equation for the derivative of the energy function resulting from differentiating the Gaussian smoothing-based approximation in Algorithm 16.15 and using Fisher's identity (Equation (16.32)).
- 16.7 Compute and plot the approximate energy function obtained for the noise variance in the model given in Exercise 7.1 by using a non-linear Kalman filter-based estimate of the energy function. You can select the filter freely.
- 16.8 Implement a random walk Metropolis-based MCMC method for estimating the noise variance in the model given in Exercise 7.1. Use one of the non-linear Kalman filters to approximate the energy function.
- 16.9 Implement a random walk Metropolis-based particle MCMC method for estimating the noise variance in the Gaussian random walk model. Use a simple bootstrap filter as the particle filter.
- 16.10 Implement a random walk Metropolis-based particle MCMC method for estimating the noise variance in the model given in Exercise 7.1.